02360323 – Project in Data Processing

Entity Resolution Dataset Construction for RelBench

Yuval Lubarsky [1]

Advisor: Prof. Benny Kimelfeld

March 2026

[1] lubarsky@cs.technion.ac.il, ID – 208001990

Contents

1 Introduction **2**

2 Background **2**
2.1 Entity Resolution . . . 2
2.2 RelBench and Relational Deep Learning . . . 3
2.3 DBLP . . . 3
2.4 Semantic Scholar . . . 3

3 Related Work **3**

4 Dataset Construction **4**
4.1 Deciding on the Dataset . . . 4
4.2 DBLP Data Extraction . . . 5
4.3 Semantic Scholar Data Extraction . . . 6
4.4 Filtering to DBLP Relevant Papers . . . 6
4.5 Ground Truth Generation . . . 7
4.6 Data Cleaning and Referential Integrity . . . 7

5 Integration with RelBench **8**
5.1 CitationDataset . . . 8
5.2 LinkageTask . . . 9

6 Proof of Concept Experiments **9**
6.1 Experimental Setup . . . 9
6.2 Graph Construction . . . 10
6.3 Task Validation . . . 10
6.4 Discussion . . . 10

7 Limitations and Future Work **10**

8 Conclusion **11**

1 Introduction

Entity Resolution (ER) is one of the oldest and most practical problems in data management. The idea is simple: given records from one or more data sources, figure out which records refer to the same real world entity. For example, the paper "Attention Is All You Need" might appear as one record in DBLP and as a different record in Semantic Scholar, with slightly different metadata, different IDs, and maybe a different title format. ER is the task of linking these two records together.

Despite decades of research, most ER benchmarks consist of small, two table datasets. Amazon vs. Google Products, Abt vs. Buy, and similar benchmarks from the Magellan repository have been the standard for years. These are useful for quick evaluation, but they do not reflect the complexity of real world data integration scenarios where you have multiple interconnected tables, millions of records, and rich relational structure (citations, authorship, venues, etc.).

Recently, the RelBench framework [Robinson et al., 2024] introduced a new paradigm: Relational Deep Learning (RDL). The idea is to represent relational databases as heterogeneous graphs and use Graph Neural Networks (GNNs) to learn directly from the multi table structure, without manual feature engineering. RelBench comes with several benchmark datasets (Amazon reviews, Stack Overflow, Formula 1, and others), but at the time of this project it did not include a large scale Entity Resolution task.

The goal of this project was to fill that gap. I built a large ER dataset by combining two major bibliographic databases: DBLP and Semantic Scholar. The dataset contains millions of papers, authors, citations, and abstracts spread across multiple relational tables. I also defined a ground truth for the matching task and integrated everything into RelBench as a custom dataset and task.

To be clear about what this project is and is not: the main contribution is the dataset construction pipeline. I did not develop new ER algorithms or train state of the art models. What I did do is all the data engineering work needed to create a usable, large scale, multi table ER benchmark, from scraping and downloading the raw data, to cleaning it, setting up a PostgreSQL database, generating ground truth, enforcing referential integrity, and packaging it for RelBench. I also ran a small proof of concept experiment on a subset of the data to show that the dataset works with RelBench's GNN and LightGBM training pipelines.

The rest of this report is organized as follows. Section 2 covers the background on ER, RelBench, and the data sources. Section 3 reviews related work. Section 4 describes the dataset construction process in detail. Section 5 explains how the dataset was integrated into RelBench. Section 6 presents the proof of concept experiment. Section 7 discusses limitations, and Section 8 concludes.

2 Background

2.1 Entity Resolution

Entity Resolution is the process of identifying records that refer to the same real world entity. It goes by many names in the literature: record linkage, deduplication, entity matching, and data matching. The general ER pipeline has two main phases:

1. **Blocking**: Reduce the number of candidate pairs by grouping records into blocks based on some cheap similarity criterion. Without blocking, you would need $O(n^2)$ comparisons which is impractical for large datasets.

2. **Matching**: For each candidate pair, determine whether the two records refer to the same entity. This can be done with rules, traditional ML classifiers, or more recently with deep learning models.

Classical approaches use string similarity measures (Jaccard, edit distance, TF-IDF) on attribute values. More recent work applies deep learning: DeepMatcher [Mudgal et al., 2018] uses attention based architectures, Ditto [Li et al., 2020] fine tunes pre-trained language models for matching, and several works explore GNN based approaches [Loster et al., 2024, Yao et al., 2022].

An important observation from Bhattacharya and Getoor [2007] is that ER can benefit from relational structure: if two papers share the same co-authors or cite the same references, that is evidence they might be

the same paper. This idea of collective ER is particularly relevant for our dataset, which has rich relational structure.

2.2 RelBench and Relational Deep Learning

RelBench [Robinson et al., 2024] is a benchmark framework for deep learning on relational databases. The core idea, introduced in Fey et al. [2024], is Relational Deep Learning (RDL): represent a relational database as a temporal heterogeneous graph, where each row in each table becomes a node, and primary key / foreign key links become edges. A GNN can then learn representations that capture the full relational structure, without any manual feature engineering.

RelBench provides:

- A `Dataset` abstraction that wraps a relational database with train/validation/test temporal splits.
- A `Task` abstraction that defines prediction tasks (node classification, regression, link prediction).
- Graph construction utilities that convert the relational schema into a PyTorch Geometric heterogeneous graph.
- Example training scripts using GNN models (HeteroGraphSAGE) and baselines (LightGBM).

Each dataset defines tables with primary keys, foreign keys, and optional time columns. The time columns enable temporal splitting: the model trains only on data up to the validation timestamp and is evaluated on future data, which prevents data leakage.

2.3 DBLP

DBLP [Ley, 2009] is a comprehensive bibliography for computer science publications. It provides structured metadata for over 6 million publications, including articles, conference papers (inproceedings), books, and theses. The data is available as a large XML dump and through a public API. Each record has a unique key (e.g., `conf/nips/VaswaniSPUJGKP17`) that serves as a persistent identifier.

For this project, I used the XML dump and parsed it into relational tables: publications, articles, inproceedings, books, incollections, authors, and an authorship relation.

2.4 Semantic Scholar

Semantic Scholar [Kinney et al., 2023] is an AI powered academic search engine maintained by the Allen Institute for AI. Its open data platform provides access to metadata for over 200 million papers from across all scientific disciplines. The data is available through a REST API and through bulk downloads of the S2ORC corpus [Lo et al., 2020].

A key feature for this project is that Semantic Scholar records often contain external identifiers, including DBLP keys, in their `externalIds` field. This cross reference is what I used to build the ground truth for the ER task.

3 Related Work

ER Benchmarks. The most commonly used ER benchmarks are relatively small. Amazon vs. Google Products, Abt vs. Buy, and the WDC Products corpus are the standard evaluation datasets. Primpeli and Bizer [2023] showed that many of these benchmarks are actually too easy for modern deep learning methods and proposed new, more challenging datasets. However, even these new datasets typically consist of just two flat tables. There is a gap in the literature for large, multi table ER benchmarks with rich relational structure.

Deep Learning for ER. DeepMatcher [Mudgal et al., 2018] was among the first to apply deep learning to entity matching. Ditto [Li et al., 2020] showed that fine tuning pre-trained language models (like BERT) can achieve strong results. HierGAT [Yao et al., 2022] uses hierarchical graph attention to model interdependence between ER decisions. GraphER [Loster et al., 2024] combines graph differential dependencies with GNNs for ER on property graphs. FlexER [Sagi et al., 2023] addresses the problem of multiple intents in ER using a multiplex graph representation, which is closely related to our setting where the same entity can appear with different roles across sources.

Blocking Methods. The blocking phase is critical for scalability. Papadakis et al. [2019] provide a comprehensive survey of blocking and filtering techniques. DeepBlocker [Thirumuruganathan et al., 2021] explores deep learning for blocking. SC-Block [Brinkmann et al., 2023] uses supervised contrastive learning for efficient blocking. Papadakis et al. [2023] provide an experimental analysis of pre-trained embeddings for both blocking and matching.

Relational Deep Learning. The RelBench benchmark [Robinson et al., 2024] and the RDL framework [Fey et al., 2024] are directly relevant to this project. Recent extensions include RelGNN [Fey et al., 2025], which introduces composite message passing specifically designed for relational database graphs.

4 Dataset Construction

This section describes the full pipeline for building the ER dataset. The process took several months and many iterations. Figure 1 shows a high level overview.

4.1 Deciding on the Dataset

Before settling on DBLP and Semantic Scholar, I spent a significant amount of time surveying existing datasets and options. I looked at:

- **Standard ER benchmarks** from Papers with Code and the Magellan repository: Amazon-Google, Abt-Buy, WDC Products. These are small (thousands of records) and consist of only two flat tables.
- **MatchBench** by Hugging Face: a collection of small two table ER datasets.
- **MovieLens + IMDB**: I considered matching movies across these two databases. MovieLens has 5 tables (ratings, tags, movies, links, genome scores) and IMDB has 7 tables. But after discussion with my advisor, we decided this might be too simple and the relational structure is not rich enough for interesting ER research.
- **Yelp vs. Google Reviews**: Another option, but with less labeled data.
- **DBLP + Citeseer**: A classic combination, but Citeseer's data quality has degraded over time.

The key insight was that we needed a dataset with two properties: (1) Multiple interconnected tables, not just a flat pair of record sets, so that RelBench's relational structure is actually useful. (2) A reliable ground truth linking mechanism.

DBLP and Semantic Scholar satisfied both requirements. Semantic Scholar has rich relational structure (papers, authors, citations, venues, abstracts) and its `externalIds` field provides a direct link to DBLP keys. An important finding from this survey was that, as far as I could tell, **there is no existing large scale multi table ER benchmark for relational data**. Most datasets in the literature are single table or two table.

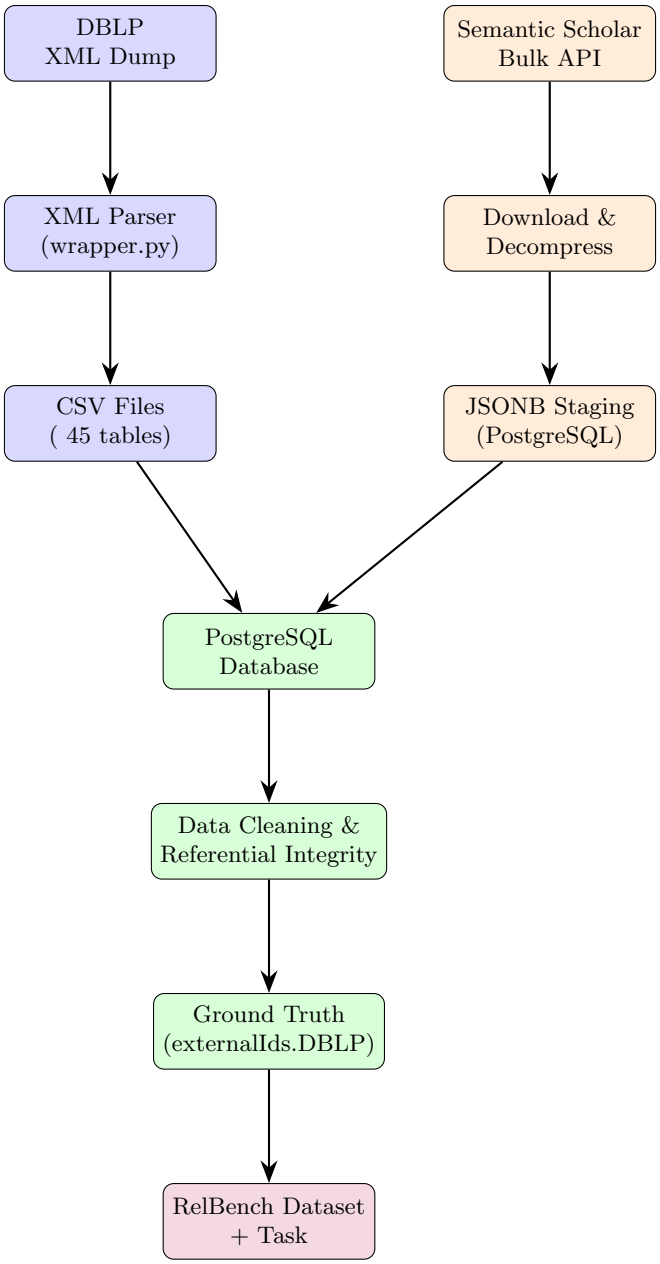

Figure 1: Overview of the dataset construction pipeline. DBLP data is parsed from XML, Semantic Scholar data is downloaded via the bulk API and staged as JSONB in PostgreSQL. Both sources are cleaned and linked through the ground truth.

4.2 DBLP Data Extraction

Extracting DBLP data was relatively straightforward. I used the XML dump (`dblp.xml`, about 4 GB) together with the DTD schema file. The parsing was done in two steps:

1. **SAX parsing** (`wrapper.py`): A streaming XML parser that reads the large XML file and produces intermediate text files (`pubFile.txt` with publication keys and types, and `fieldFile.txt` with key-value field data).

2. **CSV conversion** (`XMLToCSV.py`): Converts the intermediate files into structured CSVs with automatic

type detection (integer, float, date, string) and multi valued field handling (e.g., multiple authors per paper are pipe delimited).

This produced about 45 CSV files covering articles, inproceedings, books, incollections, authors, editors, publishers, journals, and their relationships. The main tables I used for the ER task are:

Table 1: DBLP tables used in the dataset.

Table	Rows	Description
dblp_publication	6,921,190	All publications with pubid, pubkey, title, year
dblp_article	3,412,368	Journal articles
dblp_inproceedings	3,418,148	Conference papers
dblp_book	20,360	Books
dblp_incollection	70,314	Book chapters
dblp_author	2,665,354	Author names and homepages
dblp_authored	11,403,879	Authorship relation (author × publication)

4.3 Semantic Scholar Data Extraction

Getting the Semantic Scholar data was considerably harder than DBLP. The data is available through the Semantic Scholar Academic Graph (S2AG) API, but the full corpus is enormous (over 200 million papers). I used the bulk download endpoint, which provides the data as a series of gzipped JSON files.

The process worked as follows:

1. **API discovery**: Using the S2AG API (`https://api.semanticscholar.org/datasets/v1/release`), I first identified available dataset releases and their download links. Each release contains datasets for papers, authors, abstracts, citations, paper IDs, publication venues, and TLDRs.

2. **Bulk download**: Each dataset consists of multiple shards (e.g., `papers_000.gz` through `papers_060.gz`). I wrote download scripts with retry logic (up to 7 attempts per shard, handling 502/503/504 errors) because the API was unreliable and downloads would frequently time out.

3. **JSON to CSV conversion**: Each gzipped file contains one JSON object per line. I wrote `create_csv.py` to stream through all shards, extract the relevant fields, and write them to CSV format. This was necessary because the raw JSON was too large to load into memory.

4. **PostgreSQL staging**: The converted data was loaded into PostgreSQL using JSONB columns for maximum flexibility. I used `UNLOGGED` tables during the initial load for better performance (skipping write ahead log), then migrated to logged tables.

5. **Data correction**: The CSV data needed significant cleaning before it could be loaded into PostgreSQL. I wrote correction scripts (`correct_dblp_papers.py`, `correct_dblp_authors.py`) that fixed JSON syntax issues (single quotes to double quotes, Python's `None` to JSON's `null`), formatted arrays as PostgreSQL array literals, and handled HTML entity encoding.

The total Semantic Scholar data in the database occupied over 25 GB:

All of this ran on a shared lab machine (the "TDK" server at the Technion CS department), which meant I had to be careful about disk space and memory usage. The PostgreSQL database was configured for remote access so I could work with it through SSH tunneling from Jupyter notebooks and PyCharm.

4.4 Filtering to DBLP Relevant Papers

Not all 200+ million papers in Semantic Scholar are relevant for matching against DBLP. DBLP covers computer science publications, so I filtered the Semantic Scholar data to keep only papers that have a DBLP external ID in their `externalIds` field.

Table 2: Semantic Scholar tables in the PostgreSQL database (before filtering).

Table	Rows	Storage
papers	3,162,695	4,327 MB
abstracts	–	4,577 MB
authors	–	3,936 MB
citations	6,539,402	1,734 MB
paperids	300,070	59 MB
publication_venues	–	16 KB

The filtering script (`from_papers_to_only_dblp.py`) reads the full papers CSV, parses the `externalIds` JSON field, and keeps only rows where `externalIds['DBLP']` is not null. After deduplication (keeping the record with the lowest `corpusid` when multiple records share the same DBLP key), this resulted in approximately 6.7 million matched papers.

I then created "expanded" tables that flatten the JSONB fields into proper relational columns:

Table 3: Final expanded tables from Semantic Scholar.

Table	Rows	Key Columns
papers_expanded	3,162,693	corpusid, title, year, venue, citation_count, externalid_dblp, first_author_id
authors_expanded	9,196,652	authorid, name, hindex, papercount
citations_expanded	6,539,402	citingcorpusid, citedcorpusid, isinfluential
abstracts_expanded	2,677,819	corpusid, abstract

4.5 Ground Truth Generation

The ground truth for the ER task is derived from the `externalid_dblp` column in the `papers_expanded` table. When Semantic Scholar indexes a paper, it often includes external identifiers from other databases. In particular, many records contain the DBLP key (e.g., `conf/nips/VaswaniSPUJGKP17`).

The linking logic is straightforward: a Semantic Scholar paper with `externalid_dblp = X` matches the DBLP publication with `pubkey = X`. This gives us a set of known matching pairs without any manual labeling.

There are some caveats:

- Not every Semantic Scholar paper has a DBLP ID. Only about 282,000 of the 3.16 million papers in our expanded table have this field populated.
- We treat papers without DBLP IDs as "unknown" rather than as non matches. They could be matches for which the ID is simply missing.
- The DBLP IDs in Semantic Scholar are not 100% accurate, but in practice the error rate is very low.

This ground truth approach is similar to what is commonly done in record linkage research: use existing identifier overlaps as silver standard labels. The key advantage is scale: we get hundreds of thousands of labeled pairs without any manual annotation.

4.6 Data Cleaning and Referential Integrity

Before the dataset could be used in RelBench, I needed to ensure that all foreign key references are valid. For example, if `citations_expanded` references a `citingcorpusid` that does not exist in `papers_expanded`, that row needs to be removed.

I implemented a referential integrity enforcement function that:

1. Checks that all primary keys are unique.
2. For each foreign key relationship, identifies rows where the foreign key value does not exist in the parent table.
3. Removes invalid rows and reports how many were removed.

The foreign key relationships in the dataset are:

- `dblp_article.pubid` → `dblp_publication.pubkey`
- `dblp_inproceedings.pubid` → `dblp_publication.pubkey`
- `dblp_book.pubid` → `dblp_publication.pubkey`
- `dblp_incollection.pubid` → `dblp_publication.pubkey`
- `dblp_authored.pubid` → `dblp_publication.pubkey`
- `dblp_authored.authorid` → `dblp_author.authorid`
- `abstracts_expanded.corpusid` → `papers_expanded.corpusid`
- `citations_expanded.citingcorpusid` → `papers_expanded.corpusid`
- `citations_expanded.citedcorpusid` → `papers_expanded.corpusid`
- `papers_expanded.authorId` → `authors_expanded.authorid`
- `papers_expanded.externalid_dblp` → `dblp_publication.pubkey`

The last one is the cross corpus link that defines the ER ground truth.

5 Integration with RelBench

To make the dataset usable with RelBench, I implemented two classes: a custom `Dataset` and a custom `Task`.

5.1 CitationDataset

The `CitationDataset` class (registered as `rel-citation`) extends RelBench's `Dataset` base class. Its `make_db()` method loads all the CSV files, performs the data type conversions (parsing year columns as datetime), replaces integer `pubid` values with string `pubkey` values for consistent foreign key references, and runs the referential integrity checks.

It returns a `Database` object with 11 tables, each defined with its primary key, foreign keys, and time column:

- **DBLP side**: `dblp_publication` (pkey: pubkey, time: year), `dblp_article`, `dblp_book`, `dblp_incollection`, `dblp_inproceedings`, `dblp_author` (pkey: authorid), `dblp_authored`
- **Semantic Scholar side**: `papers_expanded` (pkey: corpusid, time: year), `authors_expanded` (pkey: authorid), `citations_expanded`, `abstracts_expanded`

The temporal splits are set as:

- Validation timestamp: January 1, 2008
- Test timestamp: January 2, 2014

These were chosen so that the training set covers the historical bulk of the data, the validation set covers a six year window, and the test set covers the most recent period.

5.2 LinkageTask

The `LinkageTask` class defines the actual ER prediction task. It is registered as `citation-cs-linkage` and extends RelBench's `RecommendationTask` (which corresponds to link prediction).

The task formulation is:

> Given a paper in `papers_expanded`, predict which entries in `dblp_publication` it should be linked to.

Technically this is framed as a link prediction problem:

- Source entity: `externalid_dblp` in `papers_expanded`
- Destination entity: `pubkey` in `dblp_publication`
- Time window: 6 years (`timedelta = 365*6 days`)
- Evaluation metrics: precision, recall, and MAP at $k = 12$

The `make_table()` method generates the task labels using a SQL query (via DuckDB) that joins `papers_expanded` with `dblp_publication` on the ground truth link (`externalid_dblp = pubkey`) within the temporal window.

6 Proof of Concept Experiments

To verify that the dataset actually works with RelBench's training pipeline, I ran a small proof of concept experiment on a sampled subset of the data.

6.1 Experimental Setup

Running the full dataset requires significant compute resources (the original data needs more than 128 GB of RAM to load, and the lab machine had about 40 GB available). So I created a small subset by sampling 5,000 papers from `papers_expanded` that have valid DBLP matches (positive examples) plus 2,500 papers without matches (distractors), together with 10,000 DBLP publications (5,000 matched + 5,000 distractors). All related rows from the other tables (authors, citations, abstracts, authorship) were included while maintaining referential integrity.

After the referential integrity checks, the POC dataset contained:

Table 4: POC subset table sizes after referential integrity enforcement.

Table	**Rows**
dblp_publication	6,840
dblp_article	5,971
dblp_inproceedings	3,873
dblp_author	18,435
dblp_authored	19,248
papers_expanded	2,110
authors_expanded	5,241
citations_expanded	17
abstracts_expanded	246

6.2 Graph Construction

The key result from the POC is that the dataset successfully converts into a heterogeneous graph via RelBench's `make_pkey_fkey_graph()` function. The resulting graph has:

- **11 node types**: one for each table in the database (dblp_publication, papers_expanded, dblp_author, authors_expanded, etc.)
- **22 edge types**: one forward and one reverse edge for each foreign key relationship. For example, the `papers_expanded.externalid_dblp` → `dblp_publication.pubkey` foreign key creates edges of type `(papers_expanded, f2p_externalid_dblp, dblp_publication)` and the reverse.

The graph was built in under 1 second on the POC subset. This is the structure that a GNN would use to learn representations: it can propagate information along citation edges, authorship edges, and importantly along the cross corpus edges that connect Semantic Scholar papers to DBLP publications.

6.3 Task Validation

The `LinkageTask` successfully generated training labels (298 entity matching pairs in the training split). The temporal splitting worked correctly: the training set contains only matches where both the Semantic Scholar and DBLP records have publication dates before the validation timestamp (2008).

The validation and test splits were empty on this small subset because of the combination of temporal filtering and the small sample size. This is expected: with only 2,110 papers total (after RI filtering) and a 6 year time window, there are not enough post-2008 and post-2014 records in the sample to populate these splits. On the full dataset with millions of papers, all splits would be well populated.

6.4 Discussion

The proof of concept demonstrates that the dataset design is correct and compatible with RelBench's full pipeline:

1. **Data loading**: All 11 tables load correctly with proper types.
2. **Referential integrity**: The FK/PK relationships are valid and consistent.
3. **Graph construction**: The heterogeneous graph has the expected structure with cross corpus edges.
4. **Temporal splitting**: The train/val/test mechanism works as designed.
5. **Task generation**: The linkage task produces correct matching pairs from the ground truth.

A full scale experiment would require a machine with at least 128 GB of RAM (or implementing a streaming/chunked loading approach). With proper resources, one could train HeteroGraphSAGE or Rel-GNN models and evaluate whether the relational structure (citations, co-authorship) actually helps with entity matching beyond simple feature comparison.

7 Limitations and Future Work

Ground truth coverage. The ground truth depends on Semantic Scholar having the DBLP key in its `externalIds` field. Only about 282,000 out of 3.16 million papers have this field, so there are many potential matches that we simply do not know about. This means our evaluation metrics are conservative: a model might correctly identify a match that our ground truth does not cover, and that would count as a false positive.

Hardware constraints. The full dataset requires over 128 GB of RAM to load into memory for graph construction. The TDK lab machine had about 40 GB available, which was enough for data processing and the PostgreSQL database but not enough for full scale GNN training. Running experiments on the complete dataset would require a machine with more memory, or implementing an out of core loading strategy.

No model development. This project focused entirely on dataset construction. I did not develop or evaluate new ER models. The proof of concept experiment used RelBench's default models without any tuning. A natural next step would be to design ER specific architectures that exploit the cross corpus structure.

Author and venue matching. The current ground truth only covers paper level matching. Author disambiguation and venue matching are also important ER tasks that this dataset could support, but I did not address them in this project.

Blocking. For a production ER system, blocking would be essential to handle the scale. The dataset could be used to evaluate blocking methods (like SC-Block [Brinkmann et al., 2023] or DeepBlocker [Thirumuruganathan et al., 2021]), but I did not implement any blocking experiments.

Future work. The most immediate next steps would be:

1. Run full scale experiments on a machine with sufficient RAM.
2. Evaluate GNN models that are specifically designed for relational data (like RelGNN [Fey et al., 2025]).
3. Add blocking evaluation as a separate task.
4. Explore intentional degradation of identifiers to test robustness, for instance by removing or corrupting the DBLP keys for some records and seeing if the model can still find the matches from the relational structure alone.
5. Extend the ground truth to cover author and venue matching.

8 Conclusion

In this project, I constructed a large scale, multi table Entity Resolution dataset by combining DBLP and Semantic Scholar. The dataset contains millions of papers, authors, citations, and abstracts organized in a relational schema with well defined primary and foreign keys. The ground truth was generated from Semantic Scholar's external identifier cross references to DBLP.

I integrated the dataset into the RelBench framework as a custom dataset (`rel-citation`) and task (`citation-cs-linkage`), and verified that RelBench's GNN and LightGBM training pipelines work correctly on a proof of concept subset.

The main contribution of this project is the dataset itself and the data engineering pipeline behind it. As far as I know, this is one of the first large scale multi table ER benchmarks designed for relational deep learning. I hope it will be useful for future research on entity resolution with graph neural networks.

References

Indrajit Bhattacharya and Lise Getoor. Collective entity resolution in relational data. volume 1, pages 5:1–5:36, 2007.

Alexander Brinkmann, Roee Shraga, and Christian Bizer. Sc-block: Supervised contrastive blocking within entity resolution pipelines. *arXiv preprint arXiv:2306.09836*, 2023.

Matthias Fey, Jan Eric Lenssen, Jure Leskovec, et al. Relational deep learning: Graph representation learning on relational databases. *arXiv preprint arXiv:2312.04615*, 2024.

Matthias Fey et al. Relgnn: Composite message passing for relational deep learning. *arXiv preprint arXiv:2502.13860*, 2025.

Rodney Kinney, Chloe Anastasiades, Russell Authur, et al. The semantic scholar open data platform. *arXiv preprint arXiv:2301.10140*, 2023.

Michael Ley. Dblp: Some lessons learned. *Proceedings of the VLDB Endowment*, 2(2):1493–1500, 2009.

Yuliang Li, Jinfeng Li, Yoshihiko Suhara, et al. Deep entity matching with pre-trained language models. *Proceedings of the VLDB Endowment*, 14(1):50–60, 2020.

Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel S. Weld. S2orc: The semantic scholar open research corpus. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4969–4983, 2020.

Michael Loster, Ioannis Koumarelas, and Felix Naumann. When gdd meets gnn: A knowledge-driven neural connection for effective entity resolution in property graphs. In *Proceedings of the International Conference on Management of Data*, 2024.

Sidharth Mudgal, Han Li, Theodoros Rekatsinas, et al. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 19–34, 2018.

George Papadakis, Ekaterini Ioannou, Claudia Niederée, et al. Blocking and filtering techniques for entity resolution. *ACM Computing Surveys*, 52(4):1–42, 2019.

George Papadakis, George Mandilaras, Luca Gagliardelli, et al. Pre-trained embeddings for entity resolution: An experimental analysis. *Proceedings of the VLDB Endowment*, 16(9):2225–2238, 2023.

Anna Primpeli and Christian Bizer. A critical re-evaluation of benchmark datasets for (deep) learning-based matching algorithms. *arXiv preprint arXiv:2307.01231*, 2023.

Joshua Robinson, Matthias Fey, Daniel Zügner, et al. Relbench: A benchmark for deep learning on relational databases. In *NeurIPS Datasets and Benchmarks Track*, 2024.

Tomer Sagi, Batya Kenig, Avigdor Gal, and Benny Kimelfeld. Flexer: Flexible entity resolution for multiple intents. *Proceedings of the ACM on Management of Data*, 1(1), 2023.

Saravanan Thirumuruganathan, Han Li, Nan Tang, et al. Deep learning for blocking in entity matching: A design space exploration. In *Proceedings of the VLDB Endowment*, volume 14, pages 2459–2472, 2021.

Dezhong Yao, Yuhong Gu, Gao Cong, et al. Entity resolution with hierarchical graph attention networks. In *Proceedings of the 2022 International Conference on Management of Data*, 2022.